

Assignment 2

CS6886: Systems Engineering for Deep Learning

Nishanth Senthil Kumar

Roll No.: EE23B049

Indian Institute of Technology Madras

September 8, 2026

Question 1

(a)

Answer. I used a pretrained model, which was pretrained on ImageNet. CIFAR-10 images were resized from 32×32 to 96×96 to better match the spatial scale expected by the ImageNet-pretrained MobileNet-v2 while limiting computational overhead. The training transforms were

`Resize(96) → RandomCrop(96, padding=4) → RandomHorizontalFlip() → ToTensor() → Normalize.`

Since ImageNet-pretrained weights were used, ImageNet normalization was applied:

$$\mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225).$$

For testing, only resizing, tensor conversion, and normalization were applied.

(b)

Answer.

We used the standard torchvision MobileNet-v2 with ImageNet-pretrained weights and width multiplier 1.0. The final 1000-class classifier was replaced by a 10-class linear layer for CIFAR-10. The default MobileNet-v2 dropout and BatchNorm layers were retained without modification.

Training used cross-entropy loss and SGD with

$$\text{LR} = 0.01, \quad \text{momentum} = 0.9, \quad \text{weight decay} = 5 \times 10^{-4}.$$

A cosine annealing learning-rate schedule was used over 30 epochs with batch size 128.

(c)

Answer.

The best FP32 baseline achieved a test top-1 accuracy of

$$\boxed{95.11\%}.$$

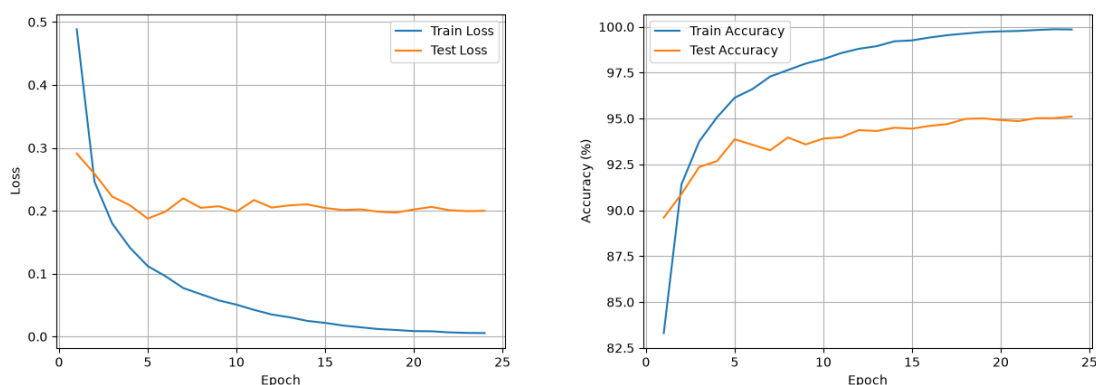


Figure 1: Training and test loss and accuracy curves for the MobileNet-v2 baseline. The curves are shown up to epoch 24, corresponding to the best checkpoint selected by test accuracy.

The main failure mode was confusion between visually similar classes. In particular, *cat* had the lowest class-wise accuracy (88.8%), followed by *dog* (90.4%). The most frequent errors were *cat*→*dog* (63 samples) and *dog*→*cat* (59 samples). Other notable confusions included *truck*→*automobile* and *bird*↔*deer*, indicating that most errors occur between semantically or visually similar classes.

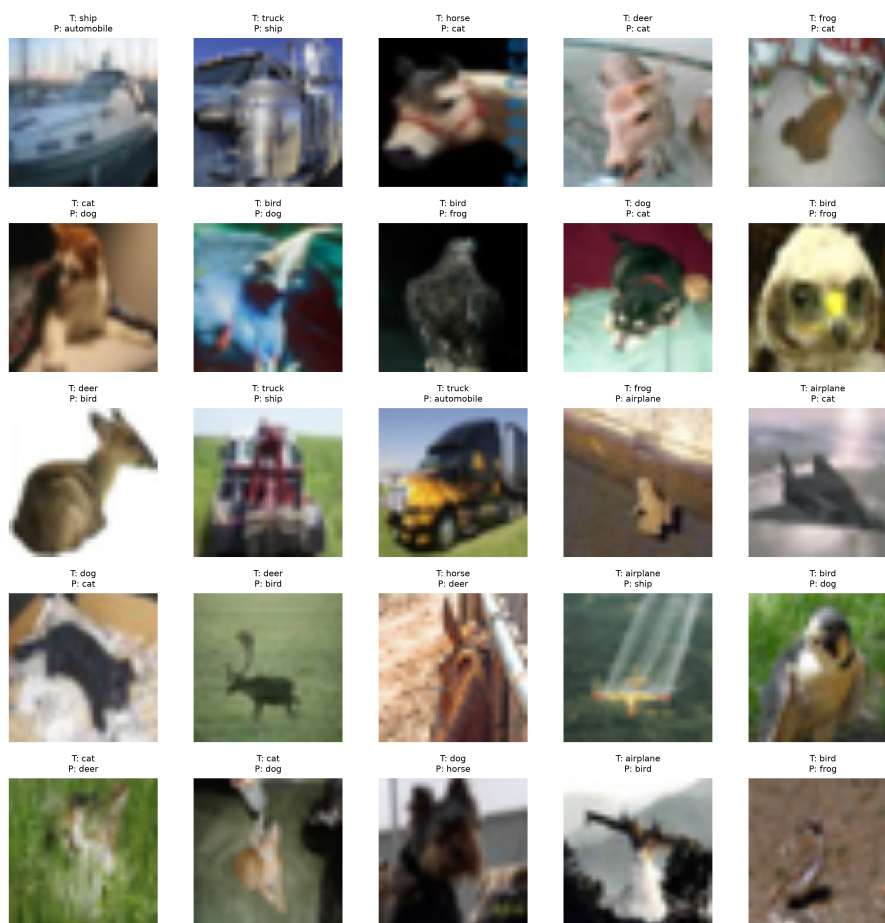


Figure 2: Representative misclassified CIFAR-10 test samples. T denotes the true class and P denotes the model-predicted class.

Question 2

(a)

Answer. We implemented a configurable compression framework combining quantization, width reduction, knowledge distillation, BatchNorm folding, and Huffman coding. We use $\mathbf{W}x\mathbf{A}y$ to denote x -bit weights and y -bit activations; for example, W6A8 uses 6-bit weights and 8-bit activations. **KD- α** denotes a knowledge-distilled MobileNet-v2 student with width multiplier α , where reducing α reduces the number of channels and hence the number of parameters.

For a signed b -bit tensor, symmetric quantization is performed using

$$q_{\max} = 2^{b-1} - 1, \quad s = \frac{\max |x|}{q_{\max}},$$

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right), -q_{\max}, q_{\max} \right), \quad \hat{x} = sq.$$

Weights are quantized **per output channel**, so each output channel has its own scale, while activations are quantized **per tensor**. A8 was used in the final configurations, corresponding to approximately $4\times$ activation compression relative to FP32.

For the high-accuracy model, layer-wise sensitivity was first measured by quantizing individual layers and recording the resulting accuracy degradation. A greedy search then assigned W5, W6, or W7 to each layer, accepting a lower precision only if the complete model remained above 94% accuracy. This gives more bits to sensitive layers and fewer bits to insensitive layers.

For higher compression, smaller MobileNet-v2 students with width multipliers 0.35 and 0.25 were trained using **knowledge distillation (KD)**, where the original 95.11% MobileNet-v2 acts as the teacher. The most aggressive configuration additionally uses **quantization-aware training (QAT)**, where quantization is simulated during training so that the network adapts to W3A8 precision.

Finally, Huffman coding is applied to the quantized weight symbols. Frequently occurring symbols receive shorter binary codes and rare symbols receive longer codes, providing additional *lossless* storage compression without changing the quantized model predictions.

(b)

Answer. Weight quantization is applied to all **Conv2d** and **Linear** layers, including the classifier. Activation quantization is applied to the outputs of these layers. In the standard mixed-precision models, BatchNorm (BN) parameters and other parameters outside the quantized weight tensors are retained as FP32 exceptions.

Three representative compressed operating points were obtained:

Table 1: Accuracy-compression trade-off of the proposed configurations.

Configuration	Accuracy (%)	Size (MB)	Compression
FP32 baseline	95.11	8.5323	1.00 $\times$
Mixed W5/W6/W7 + A8	94.04	1.7073	5.00 $\times$
KD-0.35 + W6A8 + Huffman	90.36	0.3381	25.24 $\times$
KD-0.25 + W3A8 QAT + BN fold + Huffman	88.08	0.0835	102.16 $\times$

The first configuration retains the full-width MobileNet-v2 and uses sensitivity-aware mixed precision. The second reduces the network width to 0.35, trains the student using KD, quantizes it to W6A8, and Huffman-codes the quantized weights. Huffman coding reduces its dense W6 size from 0.3631 MB (23.50 $\times$) to 0.3381 MB (25.24 $\times$), with no additional accuracy loss.

The most aggressive model uses a KD-0.25 student followed by W3A8 QAT. Before storage, 52 convolution–BatchNorm pairs are folded. For a convolution

$$y = Wx + b$$

followed by BatchNorm

$$y_{\text{BN}} = \gamma \frac{y - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

the two operations can be replaced at inference by a single convolution with

$$W' = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} W, \quad b' = \frac{\gamma(b - \mu)}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

Thus, the separate BatchNorm operation and its parameters no longer need to be stored or executed. In our model, folding all 52 BN layers changed accuracy only from 88.15% to 88.12%.

The folded W3 weights are then Huffman-coded, while the folded biases are quantized to INT16. The final accuracy is 88.08% at $102.16\times$ compression.

(c)

Answer. All reported model sizes include the storage overhead required to reconstruct the compressed model. For dense mixed-precision quantization, storage is

$$S_{\text{model}} = \sum_l N_l b_l + 32N_{\text{scales}} + 32N_{\text{FP32}},$$

where N_l is the number of weights in layer l , b_l is its assigned bit-width, N_{scales} is the number of per-channel quantization scales, and N_{FP32} is the number of parameters retained in FP32. Model size in MB is obtained by dividing the total by $8(1024)^2$.

For the KD-0.35 W6A8 model, Huffman coding gives an average weight representation of 5.3528 bits/weight instead of the dense 6 bits/weight. Its storage is

Table 2: Storage breakdown of the KD-0.35 W6A8 Huffman model.

Component	Size (MB)
Huffman weight stream	0.2520
Huffman metadata	0.005469
Quantization scales	0.0269
FP32 exceptions	0.0537
Total	0.3381

This gives

$$\frac{8.5323}{0.3381} \approx \boxed{25.24\times}$$

compression while retaining 90.36% accuracy.

For the extreme KD-0.25 model, BN folding eliminates separate BN storage and the W3 weights are Huffman-coded. FP16 scales and INT16 folded biases are used to further reduce overhead:

Table 3: Storage breakdown of the 102.16× model.

Component	Size (MB)
Huffman weight stream	0.062568
Huffman metadata	0.000486
FP16 weight scales	0.010181
INT16 folded biases	0.010181
FP16 bias scales	0.000101
Total	0.083518

Huffman coding reduces the W3 weight stream to an average of 2.1846 bits/weight. Including all metadata, scales, and biases, the final model occupies only 0.083518 MB, giving

$$\frac{8.5323}{0.083518} = \boxed{102.16\times}$$

compression at 88.08% test accuracy.

For activations, A8 reduces storage relative to FP32 by approximately

$$\frac{32}{8} = 4 \times .$$

Question 3

(a)

Answer.

To systematically study the accuracy–compression trade-off, we performed a controlled compression sweep on the original full-width MobileNet-v2 baseline. This experiment is separate from the more aggressive KD-0.35 and KD-0.25 models reported in Question 2; instead, it explores the compression regime used to obtain the high-accuracy approximately 5× compressed model.

We varied three compression parameters:

$$b_W \in \{5, 6, 7, 8\}, \quad b_A \in \{7, 8\}, \quad p \in \{0, 0.1, 0.2, 0.3\},$$

giving a total of

$$4 \times 2 \times 4 = 32$$

configurations.

Here, b_W is the weight bit-width, b_A is the activation bit-width, and p is the global magnitude-pruning ratio. We use the notation $WxAy$ to denote x -bit weights and y -bit activations; for example, W7A8 denotes 7-bit weights and 8-bit activations.

For every configuration, the same FP32 (32-bit floating-point) baseline checkpoint was reloaded. Global magnitude pruning was first applied, followed by per-output-channel weight quantization and per-tensor activation quantization. The compressed model was then evaluated on the CIFAR-10 test set. Model size, model compression ratio, activation compression ratio, sparsity, and test accuracy were recorded.

This controlled sweep isolates the effect of bit-width and pruning while keeping the underlying network architecture fixed.

(b)

Answer.

The 32 configurations were logged to Weights & Biases (W&B). Figure 3 shows the resulting Parallel Coordinates chart, with weight bit-width, activation bit-width, pruning ratio, model compression ratio, and test accuracy as the principal axes.

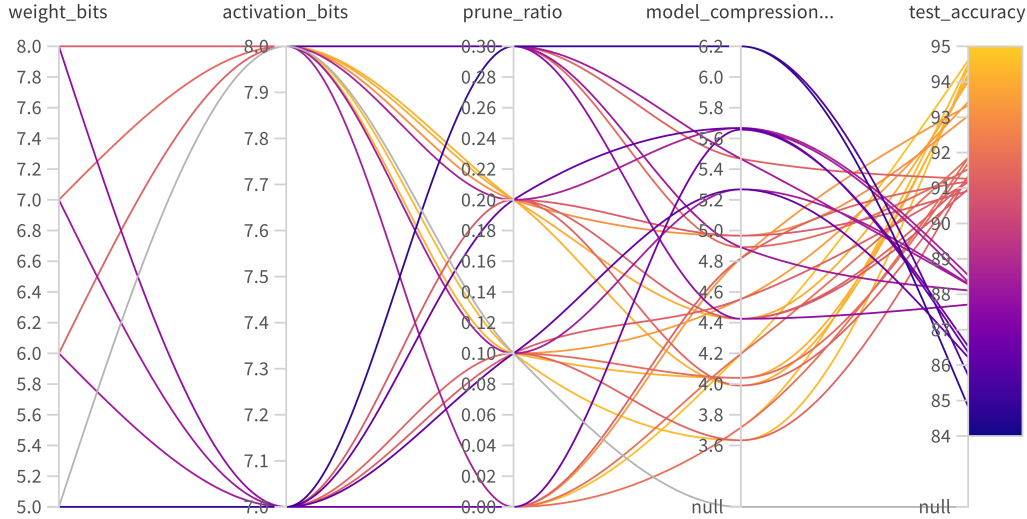


Figure 3: W&B Parallel Coordinates chart for the 32 compression configurations. $WxAy$ denotes x -bit weights and y -bit activations, while the pruning ratio denotes the fraction of weights removed by global magnitude pruning.

The sweep demonstrates the expected accuracy–compression trade-off. For example, W8A8 without pruning achieved 94.49% accuracy at $3.72\times$ model compression, while W7A8 without pruning achieved 94.60% at $4.20\times$. Increasing compression to W6A8 with 20% pruning produced 93.04% accuracy at $4.97\times$, whereas the more aggressive W5A8 with 30% pruning reached $6.20\times$ compression but reduced accuracy to 85.71%.

Activation precision was also important. For example, at W7 with no pruning, increasing activation precision from A7 to A8 improved accuracy from 91.85% to 94.60%. Consequently, A8 was retained for the final high-accuracy compression configuration.

The sweep therefore shows that uniformly reducing precision or increasing pruning eventually causes substantial accuracy degradation. This motivated the sensitivity-aware mixed-precision approach used for the approximately $5\times$ model in Question 2, where individual layers are assigned W5, W6, or W7 according to their sensitivity while activations remain A8.

Question 4

(a)

Answer.

We select the **KD-0.35 + W6A8 + Huffman** configuration as the final operating point, since it provides a strong trade-off between compression and accuracy.

The original FP32 MobileNet-v2 occupies 8.5323 MB, while the final compressed model occupies 0.3381 MB. Including all required storage overheads, the effective model-storage compression ratio (used here as the reported weight-compression ratio) is

$$\frac{8.5323}{0.3381} \approx 25.24\times.$$

The W6 weights are first represented using 6-bit quantization and then Huffman coded. The resulting Huffman stream requires only 5.3528 bits per weight on average.

(b)

Answer.

Activations are quantized to 8 bits (A8). Activation compression was measured during inference by attaching hooks to the outputs of every `Conv2d` and `Linear` layer. For one inference batch, the number of activation elements was counted and the FP32 storage,

$$32N,$$

was compared with the A8 storage,

$$8N + 32N_{\text{tensors}},$$

where one 32-bit quantization scale is included for each activation tensor.

This gives an activation compression ratio of approximately

$$4.00\times.$$

(c)

Answer.

The selected compressed model achieves a CIFAR-10 test top-1 accuracy of

$$90.36\%.$$

Compared with the 95.11% FP32 baseline, this corresponds to an accuracy reduction of 4.75 percentage points while achieving over 25 $\times$ model compression.

(d)

Answer.

The final compressed model size, including all storage overheads, is

$$0.3381 \text{ MB}.$$

Its storage consists of a 0.2520 MB Huffman weight stream, 0.005469 MB of Huffman metadata, 0.0269 MB of quantization scales, and 0.0537 MB of remaining FP32 parameters.

Question 5

(a)

Answer.

The codebase was organized into separate modules for training, evaluation, compression, experimentation, and analysis. The `baseline/` directory contains CIFAR-10 data loading, MobileNet-v2 definition, training, and evaluation. Reusable compression methods, including weight and activation quantization, pruning, layer-sensitivity analysis, and mixed-precision search, are contained in `compression/`. Knowledge distillation (KD) and quantization-aware

training (QAT) are implemented separately in `distillation/`, while complete experimental pipelines are placed in `experiments/`. Evaluation scripts are provided in `tests/`, with generated numerical results and figures stored separately in `results/` and `figures/`, respectively.

The principal scripts can be executed from the repository root using Python's module interface. For example, the FP32 baseline can be trained and evaluated using

```
python -m baseline.train
python -m tests.test_baseline
```

The compression sweep is executed using

```
python -m experiments.run_sweep
```

and the selected KD-0.35 + W6A8 + Huffman configuration is evaluated using

```
python -m experiments.huffman_distilled_w035
```

The extreme KD-0.25 + W3A8 QAT configuration can be trained and evaluated using

```
python -m experiments.train_w025_kd_qat
python -m experiments.analyze_w025_w3_bn_huffman
```

(b)

Answer.

A `README.md` is provided with the repository structure, environment setup, exact commands for training and evaluation, compression experiments, expected outputs, and checkpoint requirements. Experiments were run using Python 3.12.2, PyTorch 2.14.0, torchvision 0.29.0, NumPy 2.5.2, pandas 3.0.5, Matplotlib 3.11.1, and Weights & Biases (W&B) 0.29.0. These dependencies are pinned in `requirements.txt`.

For reproducibility, a fixed random seed of

SEED = 42

is specified in `baseline/config.py` and is used to initialize the Python, NumPy, and PyTorch random number generators. Model checkpoints and the CIFAR-10 dataset are excluded from GitHub as generated/downloaded artifacts; the README specifies the required training commands before checkpoint-dependent evaluation scripts are executed.

(c)

Answer.

The complete implementation, experiment scripts, results, figures, dependency specification, and reproduction instructions are available at:

https://github.com/lazarsalt1441/CS6886_Assignment2